FULL STACK INTERN

Technical Assessment

Task & Team Management Platform

Duration	Role
1 Day	**Full Stack Intern**

OVERVIEW

This assessment evaluates your ability to design, build, and deploy a complete full-stack web application within a single day. You will build a Task & Team Management Platform featuring authentication, backend APIs, a database layer, and a responsive user interface.

OBJECTIVE

Build and deploy a Task & Team Management Platform with authentication, backend APIs, a database, and a responsive UI. All three tiers must be live and connected:

- Frontend
- Backend
- Database

TECH STACK

Frontend	Backend	Database	Deployment
• React • Vite • React Router • Axios • Redux Toolkit • Tailwind CSS	• Node.js • Express	• MongoDB Atlas	• Frontend → Vercel • Backend → Render • Database → MongoDB Atlas

FUNCTIONAL REQUIREMENTS

1. Authentication

Implement a complete authentication flow:

- Register
- Login
- JWT authentication
- Protected routes
- Logout
- Remember Me

> **Requirement:** Passwords must be hashed using bcrypt

2. Dashboard

Build the core layout and summary view:

- Sidebar
- Navbar
- Dashboard cards

Dashboard cards must display the following metrics:

Total Tasks	Pending	Completed	In Progress

3. Task Module

Implement full CRUD operations — Create, Edit, Delete, and View Details.

Each task must contain the following fields:

- Title
- Description
- Priority
- Due Date
- Status
- Assigned User

4. Search & Filters

- Search by title
- Filter by status
- Filter by priority
- Sort by date

REST APIS

All requests and responses use JSON. Protected routes must validate the JWT.

Method	Endpoint	Purpose
POST	/register	Create a new user account
POST	/login	Authenticate a user and return a JWT
GET	/tasks	Retrieve all tasks
GET	/tasks/:id	Retrieve a single task by ID
POST	/tasks	Create a new task
PUT	/tasks/:id	Update an existing task
DELETE	/tasks/:id	Delete a task

DATABASE

Store Users and Tasks in MongoDB Atlas. Create proper Mongoose models with appropriate field types, validation, and relationships (tasks referencing their assigned user).

REQUIRED REACT CONCEPTS

Your frontend must demonstrate meaningful use of the following:

Hooks & Performance	State & Loading	Validation
• useState • useEffect • useMemo • useCallback • React.memo • Custom Hooks	• Context or Redux • Lazy Loading • Suspense	• Required fields • Email validation • Password validation • Duplicate users • Invalid JWT

ERROR HANDLING

Handle failure cases gracefully and return meaningful responses:

Code	Scenario
404	Resource not found
400	Invalid API requests
Network	Network errors
401	Unauthorized access

EXPECTED FOLDER STRUCTURE

client/src/	server/
├── components/ ├── pages/ ├── hooks/ ├── services/ ├── utils/ ├── store/	├── controllers/ ├── models/ ├── routes/ ├── middleware/ ├── config/

BONUS FEATURES

Implement any four of the following for extra credit:

Options (A)	Options (B)
• Dark Mode • Pagination • Infinite Scroll • Charts • Drag & Drop Tasks • File Upload	• Toast Notifications • Email Verification • PWA • Docker • Unit Tests

EVALUATION CRITERIA

Criteria	Marks
React Fundamentals	20
Backend APIs	20
Database Design	15
Authentication	15
Code Quality	10
UI / UX	10
Deployment	5
Documentation	5
Total	100

SUBMISSION CHECKLIST

Submit the following:

☐	GitHub repository link
☐	Live frontend URL (Vercel)
☐	Live backend URL (Render)
☐	README with setup, docs, screenshots, live URLs
☐	API collection (Postman or Bruno)
☐	**Login credentials for the deployed website (see below)**

LOGIN CREDENTIALS (DEPLOYED WEBSITE)

Provide working test credentials for the evaluator to access your deployed application. Ensure these accounts exist and are functional before submission.

Test Account — Standard User

Email	testuser@example.com
Password	Test@1234
Role	User

Test Account — Admin User (if applicable)

Email	admin@example.com
Password	Admin@1234
Role	Admin

Note: Replace the above with actual credentials before submission. Both accounts must be pre-seeded in your database and fully functional on the live deployment.

README REQUIREMENTS

Include a README in your repository covering:

- Setup instructions (local development)
- Tech stack used
- API documentation (all endpoints, request/response format)
- Screenshots of the running application
- Folder structure overview
- Live demo URLs (frontend + backend)
- Login credentials for the deployed website